C++Day 5

Template meta Programming

its nothing but doing the things in compile time rather than run time

```
int fun(5){
    return x*x;
}
cout<<fun(5)<<endl; // this happend in run time
// how to make that exceute the same thing in complie time
template <int N>
struct fun{
    static constexpr int value=N*N;
};
cout<<fun<5 ><<endl;// this happens in complie time
// the values is replaced with 25 at the compile time itself
```

```
// to use the values which are created in compile time
// we should use the following syntax
template<int N>
struct fun{
    static constexpr int value =N*N;
};
fun<6>:: value
// in this way we can use the value
```

compile time cant execute loops it can only execute recursion calls

```
template<int N >
struct Fac{
    static constexpr int value=
    N*Fac<N-1>::value;
```

```
};
template<>
struct Fac<0>{
    static constexpr int value =1;
};
// this is the way to write the define the base case
```

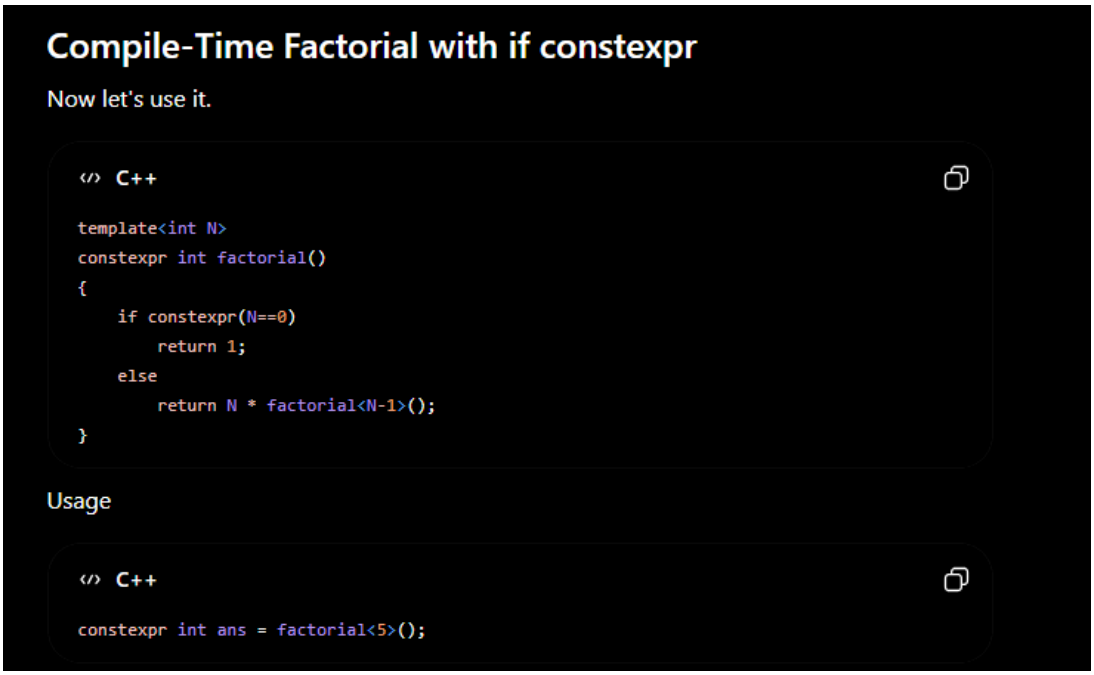
We can also use this methods to pre compute some look up tables ..

```
// complie time array generators
constexpr auto makesquare()
{
    std :: array<int ,101> table {};
    for(int i=0;i<=100;i++){
        table[i]=i*i;
    }
    return table;
}
constexpr auto squares=makesquares();
int main(){
    cout<<squares[50]<<endl;
}
// compile time genrated primes
template <int N>
constexpr auto makesieve()
{
    std::array<bool, N + 1> prime{};

    prime.fill(true);

    if constexpr (N >= 0) prime[0] = false;
    if constexpr (N >= 1) prime[1] = false;

    for (int i = 2; i * i <= N; i++)
    {
        if (prime[i])
```

```
        {
            for (int j = i * i; j <= N; j += i)
                prime[j] = false;
        }
    }

    return prime;
}
template<int N>
constexpr int countprime()
{
    constexpr auto prime=makesieve<N>();
    int cnt=0;
    for(bool x:prime){
        if(x)cnt++;
    }
    return cnt;
}
template<int N>
constexpr auto makeprimearray()
{
    constexpr auto prime=makesieve<N>();
    constexpr int cnt=countprime<N>();
    std :: array<int ,cnt> primes();
    int idx=0;
    for(int i=2;i<=n;i++){
        if(prime[i]){
            primes[idx]=i;
            idx++;
        }
    }
    return primes ;
}
constexpr auto primes=makeprimearray<100>();
int main(){
    for(auto it:primes){
```

```
        cout<<it<<" "<<endl;
    }
}
```

In the above example there is how to make effective use of this compile time calculation please use it effectively



Variadic Templates

templates that can accept any number of template arguments

```
template<typename... Ts>// here typename..ts is extension pac
k
struct Myclass{

};
// now we can define Myclass<>;Myclass<int >
//Myclass<int ,double ,char,int ,vector<int >> this is also v
alid
//typename..Ts is bag of variables
```

Function Parameter Packs

```
template<typename... Ts>//typename..Ts contians typesl like i
nt ,char
void print(Ts... args){// here Ts.. contains values
    cout<<sizeof...(args)<<endl;
}
print(10,2,3,4,5);// prints 5
// we can also write
vector<int > arg={10,20,0,6};
print(arg...) ;// compiler automaticals  genrates print(10,2
0,0,6)
```

Patterns

```
//pattern 1
args={1,2,3};
print(args..) // bcomes
==>>print(1,2,3);
//pattern 2
foo(f(args)...) // becomes
==>>foo(f(1),f(2),f(3)) //Very IMP ****
//pattern 3
foo((args+1)...)// becomes
==>>foo((1+1),(2+1),(3+1));
//pattern 4
foo(std ::move(args)...) // becomes
==>>foo(move(1),move(2),move(3));
// if Ts={int ,double ,char }
//pattern 5
tuple<Ts*...> // becomes
==>> tuple< int*,double*,char*>
//pattern 6
```

```
tuple<const Ts& ...>
==>>tuple<const int&,const double & ,const char & >
```

Variant Syntax

its same use as of a tuple

std:: variant<int ,double ,string >

Fold expressions

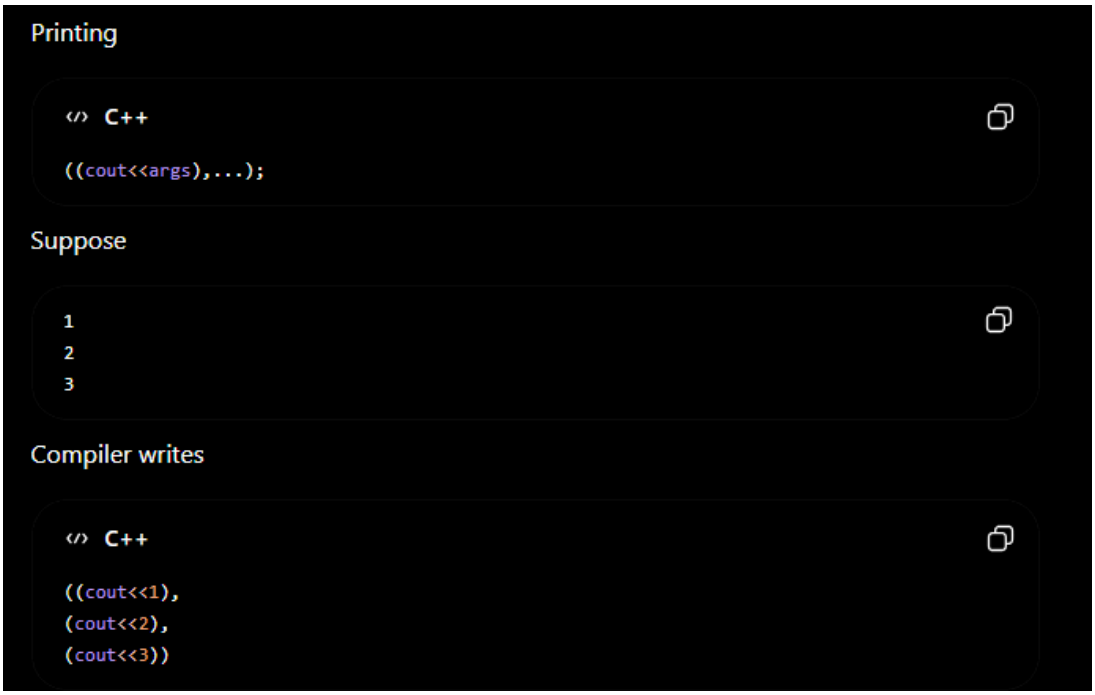
Unary Left Fold

syntax ⇒(... + args)

if args={ 1,2,3,4} ⇒ compiler generates ⇒ (((1+2)+3)+4)

```
//example
template<typename... Ts>
auto sum(Ts... args)
{
    return (... + args);
}
sum(1,2,3,4);// when we call this
(((1+2)+3)+4)// compiler genrates like this
(... && args)
args={true,true,false,true};
```

```
(((true&&true)&&false)&&true) // complier generates this
```



2. Unary Right Fold


```cpp
template<typename... Ts>
auto sum(Ts... args)
{
    return (args + ...);
}
// lets args={10,3,2};
lets apply unary left subtraction
((10-3)-2)
now unary right subtraction give
(10-(3-2))
```

3.Binary left Fold

if args={1,2,3} and init={100}

then (init +... + args) then compiler generates the following ⇒ (((100+1)+2)+3)

Example

```cpp
template<typename... Ts>
auto sum(Ts... args)
{
    return (100 + ... + args);
}
```

Call

```cpp
sum(1,2,3);
```

Compiler generates

```
(((100+1)+2)+3)

↓

106
```

4 . Binary Right Fold

if args={1,2,3} and init={100}

then (args +... + init) then compiler generates the following ⇒ 1+(2+(3+100))

```cpp
template<typename... Ts>
auto sum(Ts... args)
{
    return (args + ... + 100);
}
```

Compiler generates

```
1+(2+(3+100))

↓

106
```

SFINAE

substitution Failure is not an Error

when the compiler tries to substitute the arguments if the substitution fails do not report error just remove this overload

this was happening before c++ 20